speckit.plan

User Input

`$ARGUMENTS`

You **MUST** consider the user input before proceeding (if not empty).

Pre-Execution Checks

Check for extension hooks (before planning): - Check if `.specify/extensions.yml` exists in the project root. - If it exists, read it and look for entries under the `hooks.before_plan` key - If the YAML cannot be parsed or is invalid, skip hook checking silently and continue normally - Filter out hooks where `enabled` is explicitly `false`. Treat hooks without an `enabled` field as enabled by default. - For each remaining hook, do **not** attempt to interpret or evaluate hook `condition` expressions: - If the hook has no `condition` field, or it is null/empty, treat the hook as executable - If the hook defines a non-empty `condition`, skip the hook and leave condition evaluation to the HookExecutor implementation - For each executable hook, output the following based on its `optional` flag: - **Optional hook** (`optional: true`): ``` ## Extension Hooks

```
**Optional Pre-Hook**: {extension}
Command: `/{command}`
Description: {description}

Prompt: {prompt}
To execute: `/{command}`
```
```

- **Mandatory hook** (`optional: false`):

  ```
  ## Extension Hooks

  Automatic Pre-Hook: {extension}
  Executing: `/{command}`
  EXECUTE_COMMAND: {command}
  ```
  ```

```
Wait for the result of the hook command before proceeding to
the Outline.
```

After emitting the block above you MUST actually invoke the hook and wait for it to finish before continuing. Run it the same way you would run the command yourself in this agent/session (the invocation may differ from the literal `{command}` id shown above, e.g. a skills-mode agent runs it as `/skill:speckit-...` or `$speckit-...`). Emitting the block alone does not run the hook.

- If no hooks are registered or `.specify/extensions.yml` does not exist, skip silently

Outline

1. **Setup**: Run `.specify/scripts/bash/setup-plan.sh --json` from repo root and parse JSON for FEATURE_SPEC, IMPL_PLAN, SPECS_DIR, BRANCH. For single quotes in args like "I'm Groot", use escape syntax: e.g 'I'\''m Groot' (or double-quote if possible: "I'm Groot").

2. **Load context**: Read FEATURE_SPEC and `.specify/memory/constitution.md`. Load IMPL_PLAN template (already copied).

3. **Execute plan workflow**: Follow the structure in IMPL_PLAN template to:

   - Fill Technical Context (mark unknowns as "NEEDS CLARIFICATION")
   - Fill Constitution Check section from constitution
   - Evaluate gates (ERROR if violations unjustified)
   - Phase 0: Generate research.md (resolve all NEEDS CLARIFICATION)
   - Phase 1: Generate data-model.md, contracts/, quickstart.md
   - Re-evaluate Constitution Check post-design

Mandatory Post-Execution Hooks

You MUST complete this section before reporting completion to the user.

Check if `.specify/extensions.yml` exists in the project root. - If it does not exist, or no hooks are registered under `hooks.after_plan`, skip to the Completion Report. - If it exists, read it and look for entries under the `hooks.after_plan` key. - If the YAML cannot be parsed or is invalid, skip hook checking silently and continue to the Completion Report. - Filter out hooks where `enabled` is explicitly `false`. Treat hooks without an `enabled` field as enabled by default. - For each remaining hook, do **not** attempt to interpret or evaluate hook `condition` expressions: - If the hook has no `condition` field, or it is null/empty, treat the hook as executable - If the hook defines a non-empty `condition`, skip the hook and leave condition evaluation to the HookExecutor implementation - For each executable hook, output the following based on its `optional` flag: - **Mandatory hook** (`optional: false`) — **You MUST emit `EXECUTE_COMMAND:` for each mandatory hook**: ``` ## Extension Hooks

```
**Automatic Hook**: {extension}
Executing: `/{command}`
EXECUTE_COMMAND: {command}
```
```

```
After emitting the block above you MUST actually invoke the hook
and wait for it to finish before continuing. Run it the same way
you would run the command yourself in this agent/session (the
invocation may differ from the literal `{command}` id shown above,
e.g. a skills-mode agent runs it as `/skill:speckit-...` or
`$speckit-...`). Emitting the block alone does not run the hook.
```

- **Optional hook** (`optional: true`):

  ```
  ## Extension Hooks

  **Optional Hook**: {extension}
  Command: `/{command}`
  Description: {description}

  Prompt: {prompt}
  To execute: `/{command}`
  ```

Completion Report

Command ends after Phase 1 design. Report branch, IMPL_PLAN path, and generated artifacts.

Phases

Phase 0: Outline & Research

1. **Extract unknowns from Technical Context** above:

   - For each NEEDS CLARIFICATION → research task
   - For each dependency → best practices task
   - For each integration → patterns task

2. **Generate and dispatch research agents**:
  ```

```
For each unknown in Technical Context:
  Task: "Research {unknown} for {feature context}"
For each technology choice:
  Task: "Find best practices for {tech} in {domain}"
```

3. **Consolidate findings** in `research.md` using format:

   - Decision: [what was chosen]
   - Rationale: [why chosen]
   - Alternatives considered: [what else evaluated]

Output: research.md with all NEEDS CLARIFICATION resolved

Phase 1: Design & Contracts

Prerequisites: `research.md` complete

1. **Extract entities from feature spec** → `data-model.md`:
   - Entity name, fields, relationships
   - Validation rules from requirements
   - State transitions if applicable
2. **Define interface contracts** (if project has external interfaces) → `/contracts/`:
   - Identify what interfaces the project exposes to users or other systems
   - Document the contract format appropriate for the project type
   - Examples: public APIs for libraries, command schemas for CLI tools, endpoints for web services, grammars for parsers, UI contracts for applications
   - Skip if project is purely internal (build scripts, one-off tools, etc.)
3. **Create quickstart validation guide** → `quickstart.md`:
   - Document runnable validation scenarios that prove the feature works end-to-end
   - Include prerequisites, setup commands, test/run commands, and expected outcomes
   - Use links or references to contracts and data model details instead of duplicating them
   - Do not include full implementation code, model/service/controller bodies, migrations, or complete test suites
   - Keep this artifact as a validation/run guide; implementation details belong in `tasks.md` and the implementation phase

Output: data-model.md, /contracts/*, quickstart.md

Key rules

- Use absolute paths for filesystem operations; use project-relative paths for references in documentation

- ERROR on gate failures or unresolved clarifications

Done When

- [ ] Plan workflow executed and design artifacts generated
- [ ] Extension hooks dispatched or skipped according to the rules in Mandatory Post-Execution Hooks above
- [ ] Completion reported to user with branch, plan path, and generated artifacts